



I want to build a terminal coding tool like claude code

But here is the catch

We will be using different ai providers

Like open router

Nvidia nim

Zenmux

llama

Etc

First user need to install our coding tool using npm command

Then in c drive in users folder in current user folder our folder will be generated with name of our coding tool

In which there will be settings.json file in which you need to provide all api provider details in an array like

```
{  
  Priority: 1  
  Api_provider:"",  
  Model:"",  
  Api_key: "",  
  Read: True/False,  
  Write: True/False  
}
```

Priority will decide which api provider will be used first then next and so on when api will reach its limit or gives any error then the model will give switch to next api provider

How is the idea?



ChatGPT gpt-5-5

The idea is actually quite strong. You're essentially building a **provider-agnostic AI coding agent**, similar to Claude Code, but with a **multi-provider failover and routing system**. That's a real pain point today because developers don't want to be locked into a single AI provider.

Why this idea is good

Current tools have limitations:

Tool	Problem
Claude Code	Anthropic only
Gemini CLI	Google only
Cursor	Limited model selection
OpenAI Codex CLI	OpenAI only
Continue.dev	Supports many providers but not seamless terminal-first failover

Your idea solves:

- Vendor lock-in
- API outages
- Rate limits
- Cost optimization
- Automatic fallback

This is genuinely useful.

Architecture I'd recommend

```
text

mycode init
mycode login
mycode chat
mycode explain app.js
mycode fix error.log
mycode edit src/app.ts
mycode agent
```

Installation

```
bash

npm install -g mycode
```

After installation:

text

```
C:\Users\Ankit\.mycode\
```

or

text

```
~/.mycode/
```

for Linux/Mac.

Folder Structure

text

```
.mycode/  
|  
├── settings.json  
├── sessions/  
├── cache/  
├── logs/  
├── prompts/  
├── memory/  
├── plugins/  
└── history/
```

settings.json

I would improve your structure.

json

```
{  
  "providers": [  
    {  
      "priority": 1,  
      "provider": "openrouter",  
      "model": "anthropic/claude-sonnet-4",  
      "apiKey": "xxxx",  
      "read": true,  
      "write": true,  
      "enabled": true,  
      "maxRetries": 3  
    }  
  ]  
}
```

```
    },
    {
      "priority": 2,
      "provider": "nvidia-nim",
      "model": "llama-3.3-70b",
      "apiKey": "xxxx",
      "read": true,
      "write": false,
      "enabled": true,
      "maxRetries": 3
    },
    {
      "priority": 3,
      "provider": "ollama",
      "model": "qwen3-coder",
      "baseUrl": "http://localhost:11434",
      "read": true,
      "write": true,
      "enabled": true
    }
  ]
}
```

Why Read and Write permissions are brilliant

Example:

Read

Allowed:

- Explain code
- Analyze project
- Search files
- Generate documentation

Write

Allowed:

- Modify files
- Delete code
- Run migrations
- Refactor project

This gives security.

Claude Code doesn't have this exact provider-level permission model.

Provider Manager

typescript

```
class ProviderManager {
  providers = [];

  async getProvider() {
    sortByPriority();

    for (provider of providers) {
      try {
        await provider.ping();
        return provider;
      } catch {
        continue;
      }
    }

    throw Error("No providers available");
  }
}
```

Failover Logic

text

```
Provider 1
↓
Rate Limit
↓
Provider 2
↓
429 Error
↓
Provider 3
↓
Network Error
↓
```

This is one of the biggest selling points of your tool.

Cost Routing (even better)

Example:

json

```
{
  "purpose": "chat",
  "provider": "llama"
},
{
  "purpose": "coding",
  "provider": "claude"
},
{
  "purpose": "review",
  "provider": "gemini"
}
```

You can save users a lot of money.

Agent Permissions

When running:

bash

mycode agent

Ask:

text

Allow:

- ✓ Read Files
- ✓ Write Files
- ✓ Execute Commands
- ✓ Internet Access
- ✓ Git Access

Exactly like Claude Code.

Architecture

text

```
CLI
↓
Command Router
↓
Agent Engine
↓
Provider Manager
↓
Model Adapter Layer
↓
AI Providers
```

Adapter Pattern

Every provider should implement:

typescript

```
interface AIProvider {
  chat()
  stream()
  embeddings()
  listModels()
  healthCheck()
}
```

Then adding new providers becomes easy.

Context Engine

Need:

text

```
.gitignore parser
File chunking
Token counting
Context compression
Conversation history
```

Otherwise large projects won't work.

Features I'd add

Git Integration

```
bash

mycode commit
mycode review-pr
mycode explain-diff
```

Session Memory

```
bash

mycode save-session
mycode resume
```

Project Indexing

```
bash

mycode index
```

Creates:

```
text

.mycode/index.db
```

using SQLite.

MCP Support

This is extremely important.

Support:

- GitHub
- Figma

- Browser
- Database
- Slack
- Jira

Otherwise enterprise adoption becomes difficult.

Potential Problems

1. Different models behave differently

Claude:

text

Produces precise code edits.

Gemini:

text

Long explanations.

Llama:

text

Sometimes hallucinates.

Need normalization layer.

2. Context windows differ

text

Claude → 200k+
Gemini → 1M+
Llama → 32k

Need dynamic context management.

3. Tool calling differs

Every provider has different APIs.

Need unified schema.

Tech Stack I'd choose

```
text

Node.js
TypeScript
Commander.js
Ink
Chokidar
SQLite
Zod
LangGraph (optional)
Tree-sitter
```

Monetization

Free

- Bring your own API keys.

Pro

- Cloud sync.
- Shared memories.
- Team workspaces.
- Prompt marketplace.
- Premium providers.

Rating of the idea

Category	Score
Technical feasibility	9/10
Market need	8.5/10
Competition	7/10
Monetization	8/10

Category	Score
Uniqueness	8.5/10

Overall: 8.5/10

The multi-provider failover system combined with terminal-first coding experience is genuinely valuable.

I would go one step further and make it:

"The universal AI coding agent that never stops working because it can switch between any model automatically."

That's a strong product positioning.